Chapter 1 · Breaking the Feedback Loop

Exact block-parallel evaluation of an integer one-pole filter: the mathematics, the threshold-map algorithm, and the history-certified (H-cert) executor

1.1 Why a one-pole filter resists speed-up
1.2 The integer update and its rounding
1.3 Two facts about a single step
1.4 Where states get stuck
1.5 Block maps
1.6 How far can a block collapse?
1.7 Algorithm I: the threshold-map executor
1.8 A change of viewpoint: certifying the entry
1.9 The shared-base rank table
1.10 Algorithm II: the H-cert executor
1.11 Throughput and latency
1.12 The hardware budget on a small FPGA
1.13 Measuring what matters
1.14 What is new, and what is not
1.15 Summary · Problems · Answers

Few circuits are as simple as the one-pole filter, and few are as stubborn. It has one state, one multiply-and-add, and one register, yet the very thing that makes it useful, its memory of the past, makes it impossible to compute faster than one sample per trip around its feedback loop. This chapter develops, from first principles, a way to beat that limit **without changing a single output bit**.

The road has three stages. First, we study the integer filter itself: how its rounding works, where it gets stuck, and how a block of inputs squeezes many possible starting states into a few. Second, we use that squeezing to build a first parallel algorithm, the *threshold map*, and we find out exactly why it is too slow. Third, we change our point of view. Instead of preparing for every possible starting state, we use the inputs just before a block to *certify* a small window that must contain the true one, and we build a compact table, the *rank table*, for just that window. The result is the history-certified executor, H-cert.

Every symbol is defined before it is used, and Table 1.1 collects them. Each new idea is tested on a small worked example before it is used in an argument. Read with a pencil: the examples are small enough to check by hand, and doing so is the fastest way to make the ideas your own.

Table 1.1. Notation. The running examples use $a = 61/64$ ($m = 3$) and $a = 63/64$ ($m = 1$) with $B = 17$, and a toy filter $a = 3/4$ with $B = 4$ that is small enough to check by hand.

Symbol	Meaning
n	Sample index, $n = 1, 2, \ldots$
$x[n]$, $s[n]$	Input sample n, and the state after the n-th update. Both are non-negative integers. $s[0]$ is the initial state.
s, s'	The state before and after one update.
B, $S_{\max}$	Word width in bits, and the largest storable value, $S_{\max} = 2^B - 1$. We use $B = 17$, so $S_{\max} = 131071$.
$a = A/D$	Retention: the fraction of the old state kept each step, written as a ratio of integers $A < D$.
m	$m = D - A$. The new input receives weight $m/D = 1 - a$.
τ	Time constant, $\tau = D/m = 1/(1 - a)$.
ρ	Rounding offset, an integer with $0 \le \rho \le D - 1$.
c_n	Per-sample constant $c_n = m\,x[n] + \rho$.
q, r	Quotient and remainder of a division by D: $N = qD + r$ with $0 \le r \le D - 1$.
$\lfloor u \rfloor$, $\lceil u \rceil$	Floor (largest integer $\le u$) and ceiling (smallest integer $\ge u$).
L, P	Block length (samples per block), and number of parallel workers.
F	Block map: the exit state of a block as a function of its entry state.
K	Number of thresholds of a block map, equal to (distinct exit states) -1.
d, w	A gap between two states, and the width (largest minus smallest) of a set of states.
R	Width floor, $R = \lfloor (D - 1)/m \rfloor$.
H	History length: the number of preceding inputs used to certify a block's entry.
ℓ, h	Lower and upper rails: the states reached from 0 and from $S_{\max}$ after the history.
e	The true entry state of a block.
b, u_i	Rank-table base and lane offsets (Section 1.9).
Θ	Throughput in samples per clock. f_k is the clock frequency of a design that does k updates per clock.

1.1 Why a one-pole filter resists speed-up

1.1.1 The loop

With unlimited precision, a one-pole filter computes

$$y[n] = a\,y[n-1] + (1-a)\,x[n]. \tag{1.1}$$

Each output is a weighted average of the previous output and the new input. Because $y[n]$ needs $y[n-1]$, and $y[n-1]$ needs $y[n-2]$, the computation is a chain. Figure 1.1(a) shows the hardware: a combinational *update block* U that computes the new state, and a register that holds it. Every clock, the state leaves the register, passes through U, and returns.

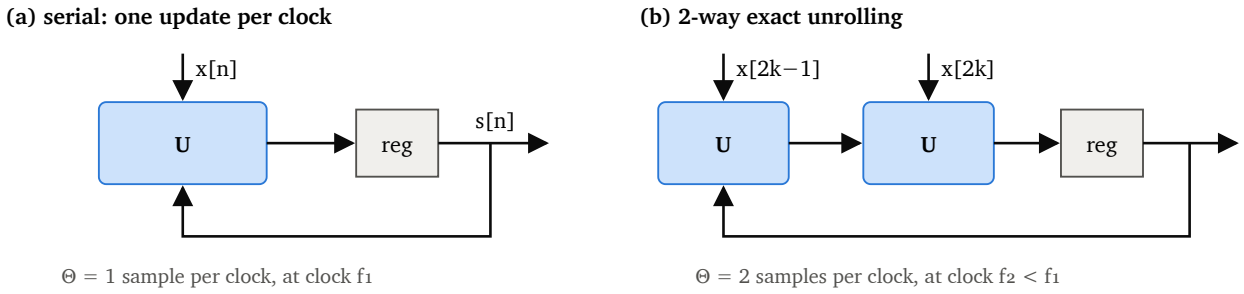


Figure 1.1. (a) The serial filter: the state must pass through the update block U and the register once per sample, so the loop delay caps throughput at one sample per clock. (b) Exact unrolling chains two copies of U so that one clock produces two samples. The loop gets longer, so the clock gets slower.

The time around this loop, $t_{\text{loop}} = t_U + t_{\text{reg}}$, sets the fastest possible clock $f_1 = 1/t_{\text{loop}}$. Since the serial design produces one sample per clock, its throughput is $\Theta = 1$ sample per clock, or f_1 samples per second. Adding more hardware beside the loop does not help: the loop itself is the bottleneck. This limit is called the *iteration bound* of the recurrence.

1.1.2 Unrolling, and why it is only a partial answer

The standard remedy is to put k copies of U in a row (Figure 1.1(b)). One clock then produces k samples, but the signal must ripple through k update blocks before the register captures it, so the clock slows to some $f_k < f_1$. The throughput in samples per second becomes kf_k. Because the register and routing overhead are shared, f_k falls more slowly than f_1/k, so unrolling does help, but with diminishing returns. In this chapter "2-way" and "4-way unrolling" mean exactly this circuit, with $k = 2$ and $k = 4$.

A natural question is whether algebra can shorten the chain. For the real-valued filter it can: substituting (1.1) into itself gives $y[n] = a^2 y[n-2] + \ldots$, a *look-ahead* form whose loop spans two samples but contains only one multiply-add. For the **integer** filter, look-ahead fails, because each step rounds, and two roundings in a row are not one rounding of the combined expression. Example 1.1 shows this concretely, after we have defined the integer update.

1.1.3 What we are after

We want a circuit that

1. produces **exactly** the same integer outputs as the serial filter, bit for bit and for every input; and
2. delivers more samples per second than the best exact unrolled design on the same device.

The obvious strategy is to cut the stream into blocks of L samples and give each block to its own worker. The obstacle is just as obvious: a worker cannot start its block until it knows the state at the end of the previous block. The rest of this chapter is about removing that obstacle.

1.2 The integer update and its rounding

1.2.1 From real numbers to integers

Hardware stores the state as a B-bit integer, and the retention must be a ratio of integers, $a = A/D$. Choosing D as a power of two (for example $D = 64$) makes division by D a right shift. Multiplying (1.1) by D gives $Dy[n] = Ay[n-1] + mx[n]$, where $m = D - A$. The right side is an integer, but dividing it by D usually is not, so the integer filter rounds:

$$s' = \left\lfloor \frac{A\,s + m\,x + \rho}{D} \right\rfloor = \left\lfloor \frac{A\,s + c}{D} \right\rfloor, \qquad c = m\,x + \rho. \tag{1.2}$$

For $a = 63/64$ this reads $s' = \lfloor (63s + x + \rho)/64 \rfloor$. In hardware, $63s = 64s - s$ is a shift and a subtraction, and $\lfloor \cdot /64 \rfloor$ is a 6-bit right shift. For $a = 61/64$, $61s = 64s - 2s - s$ costs one more subtraction. No multiplier is needed for either.

Will the result ever overflow? The numerator is at most $AS_{\max} + mS_{\max} + \rho = DS_{\max} + \rho$, and since $\rho < D$, the quotient is at most $S_{\max}$. Clipping to the range $[0, S_{\max}]$ is therefore never triggered for in-range inputs, and we omit it.

1.2.2 What the rounding offset does

Write $v = (As + mx)/D$ for the exact, unrounded result. Then (1.2) outputs $\lfloor v + \rho/D \rfloor$. The hardware always rounds *down*, but first it adds ρ/D. That choice is the rounding rule:

Table 1.2. The rounding offset selects the rounding rule.

ρ	Adds before flooring	Rule	Output $-\,v$ lies in
0	0	truncate (round down)	$(-1,\ 0]$
$D/2$	$\frac{1}{2}$	round to nearest	$(-\frac{1}{2},\ \frac{1}{2}]$
$D-1$	$1 - \frac{1}{D}$	round up ($\lceil v \rceil$)	$(-\frac{1}{D},\ 1 - \frac{1}{D}]$

Why talk about ρ at all, if we get to choose it? Because the job here is to reproduce an *existing* integer filter exactly. Whoever wrote that filter already chose its rounding rule, and "exact" means matching their choice. Writing the rule as a parameter ρ lets one analysis cover every choice. As we will see, ρ changes where the filter gets stuck near zero, so the hardware must be designed for the value actually in use.

1.2.3 The step form

Because $A = D - m$, we have $As = Ds - ms$, and the integer s can be pulled out of the floor ($\lfloor k + u \rfloor = k + \lfloor u \rfloor$ for integer k):

$$s' = s + \left\lfloor \frac{m\,(x - s) + \rho}{D} \right\rfloor. \tag{1.3}$$

Equation (1.3) says the state moves toward the input by a rounded version of $(x - s)/\tau$. When that move is smaller than the rounding can resolve, the floor is zero and the state does not move at all. Keep this picture in mind; the rest of the chapter grows out of it.

Example 1.1 — Look-ahead is not exact. Take the toy filter $a = 3/4$ ($A = 3$, $D = 4$, $m = 1$) with $\rho = 2$. Start from $s = 0$ and apply inputs $x_1 = 0$, then $x_2 = 1$, so $c_1 = 2$ and $c_2 = 3$.

Two real steps of the integer filter. $s_1 = \lfloor (0 + 2)/4 \rfloor = 0$, then $s_2 = \lfloor (0 + 3)/4 \rfloor = 0$.

One merged step. Substituting the first update into the second *before* rounding gives $\lfloor (A^2 s + A c_1 + D c_2)/D^2 \rfloor = \lfloor (0 + 6 + 12)/16 \rfloor = 1$.

The answers differ, 0 versus 1. The merged formula rounds once, but the filter rounds twice, and the first rounding throws away information (here, the 2/4 left over in step 1) that the second step never sees. Any exact speed-up must respect each individual rounding.

1.3 Two facts about a single step

Everything that follows rests on two properties of (1.2). Consider two states $s < t$ receiving the same input, so they share the same c.

Fact 1 (order is kept). $At + c > As + c$, and flooring cannot reverse an inequality, so $s' \le t'$. The update is *monotone*.

Fact 2 (the gap formula). Let $d = t - s$ be the gap. Divide the lower numerator by D: $As + c = qD + r$, so $s' = q$ and $r = (As + c) \mod D$. Since $Ad = Dd - md$,

$$At + c = qD + r + Dd - md \quad \Rightarrow \quad t' = q + d + \left\lfloor \frac{r - md}{D} \right\rfloor,$$

and so the new gap is

$$d' = d + \left\lfloor \frac{r - m\,d}{D} \right\rfloor. \tag{1.4}$$

Because $r \le D - 1$, the fraction $(r - md)/D$ is less than 1, so its floor is at most 0. **Gaps never grow.** They shrink exactly when the remainder r is small compared with md.

Equation (1.4) is the workhorse of this chapter. Notice what it says: the new gap depends only on the old gap d and on the remainder r of the *lower* state. If we know one state exactly, together with its remainder, we can track any nearby state using only the small number d. Section 1.9 turns this observation into hardware.

The case of small gaps deserves its own statement. If $md \le D - 1$, then $r - md$ lies between $-(D-1)$ and $D - 1$, so the floor in (1.4) is either 0 or -1:

$$d' = \begin{cases} d - 1, & r < m\,d, \\ d, & r \ge m\,d. \end{cases} \qquad \text{(valid when } md \le D - 1) \tag{1.5}$$

A small gap either survives unchanged or closes by exactly one, and a single comparison decides which.

> **Example 1.2 — The gap formula at work.** Toy filter 3/4, $\rho = 2$, input $x = 3$ (so $c = 5$). Take $s = 5$ and $t = 7$ ($d = 2$). Then $As + c = 20 = 5 \cdot 4 + 0$, so $s' = 5$ and $r = 0$. Since $r = 0 < md = 2$, (1.5) predicts $d' = 1$, so $t' = 6$. Check directly: $t' = \lfloor (21 + 5)/4 \rfloor = \lfloor 6.5 \rfloor = 6$. ✓

1.4 Where states get stuck

1.4.1 The zero-input dead band

Set $x = 0$ in (1.3): $s' = s + \lfloor (\rho - ms)/D \rfloor$. If $ms > \rho$, the floor is negative and the state drops. If $ms \le \rho$, the fraction lies in $[0, 1)$, the floor is 0, and the state stays where it is forever. So with zero input,

$$s \text{ is a fixed point} \Leftrightarrow s \le \frac{\rho}{m}, \qquad \text{that is} \qquad s \in \{0, 1, \ldots, \lfloor \rho/m \rfloor\}. \tag{1.6}$$

This set is the *dead band*. For 63/64 it is $\{0\}$ with truncation, $\{0, \ldots, 32\}$ with $\rho = 32$, and $\{0, \ldots, 63\}$ with $\rho = 63$.

Figure 1.2 shows what this does to a decay. Large states shrink by about s/τ per step, just like the real filter, but near the bottom the step is rounded to exactly 1, then to 0. The decay becomes a straight line and then stops dead at the top of the band.

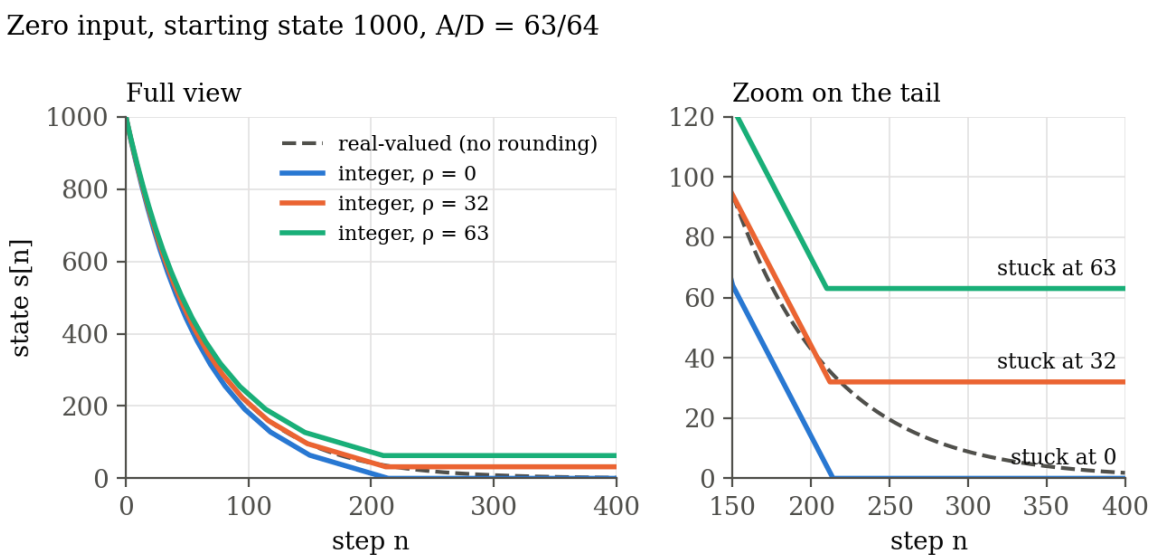


Figure 1.2. Zero-input decay from state 1000 with a = 63/64. Left: the three integer versions track the real-valued exponential while the state is large. Right: near the bottom they decay linearly and then stop. Truncation (ρ = 0) reaches 0; ρ = 32 sticks at 32 and ρ = 63 at 63.

> **A common misconception.** It is tempting to model the integer decay as $s[0]\,a^n$ plus a small constant, and conclude that a long enough block drives every state as close to the bottom as we like. The constant is real, but it never decays: it is the top of the dead band. Running longer changes *when* a state arrives there, not *where* it ends up.

1.4.2 Constant input: the band moves with the input

For any constant input x, (1.3) leaves the state fixed exactly when $0 \le m(x-s)+\rho < D$, that is

$$x - \frac{D-\rho}{m} \;<\; s \;\le\; x + \frac{\rho}{m}. \tag{1.7}$$

This is a band of length $D/m = \tau$ around the input. States above it drop, states below it rise, and states inside it never move. For 63/64 ($m = 1$), a mid-range constant input such as $x = 1000$ leaves **64 stuck states for every choice of** ρ. The zero-input dead band is simply this band with its lower part cut off at 0.

1.5 Block maps

1.5.1 Definition

Fix a block of L known inputs and run the filter through it from some entry state s. Since the inputs are fixed, the exit state depends only on s. Define the *block map*

$$F(s) = \text{the state after the } L \text{ updates of the block, entered at } s. \tag{1.8}$$

If we knew F for every block, the serial chain would become a chain of lookups. Block 1's exit is $F_1(s[0])$, block 2's is $F_2(F_1(s[0]))$, and so on. Each F_j depends only on its own block's inputs, so all of them can be built at the same time. That is the whole idea of block parallelism. The question is how to represent F cheaply.

1.5.2 The staircase

Facts 1 and 2 survive composition: a sequence of monotone, gap-non-increasing steps is still monotone and gap-non-increasing. Applied to neighbouring entries s and $s+1$ (gap 1), this gives

$$F(s+1) - F(s) \in \{0, 1\}. \tag{1.9}$$

So F is a staircase with steps of height exactly one. When $F(s+1) = F(s)$, the two entries have *collapsed*: the block has forgotten the difference between them. When $F(s+1) = F(s) + 1$, we call $s+1$ a *threshold*. Then F is described completely by its *base* $F(0)$ and its sorted list of K thresholds $t_1 < \ldots < t_K$:

$$F(s) = F(0) + \#\{\, i : t_i \le s \,\}, \qquad K = F(S_{\max}) - F(0). \tag{1.10}$$

Here $\#\{\cdot\}$ means "the number of elements of". $K+1$ is the number of distinct exit states.

Example 1.3 — A complete block map. Toy filter 3/4, $\rho = 2$, $B = 4$ (states 0–15), block inputs $x = 9, 0, 4$. Applying (1.2) three times to each entry:

s	**0**	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**	**12**	**13**	**14**	**15**
$F(s)$	3	3	3	**4**	4	**5**	5	**6**	6	6	**7**	7	7	**8**	**9**	9

Sixteen entries produce only seven exits. The map is the base $F(0) = 3$ plus the $K = 6$ thresholds $\{3, 5, 7, 10, 13, 14\}$ (bold). Check one value: $F(11) = 3 + \#\{3, 5, 7, 10\} = 7$. ✓ Figure 1.3 draws the staircase.

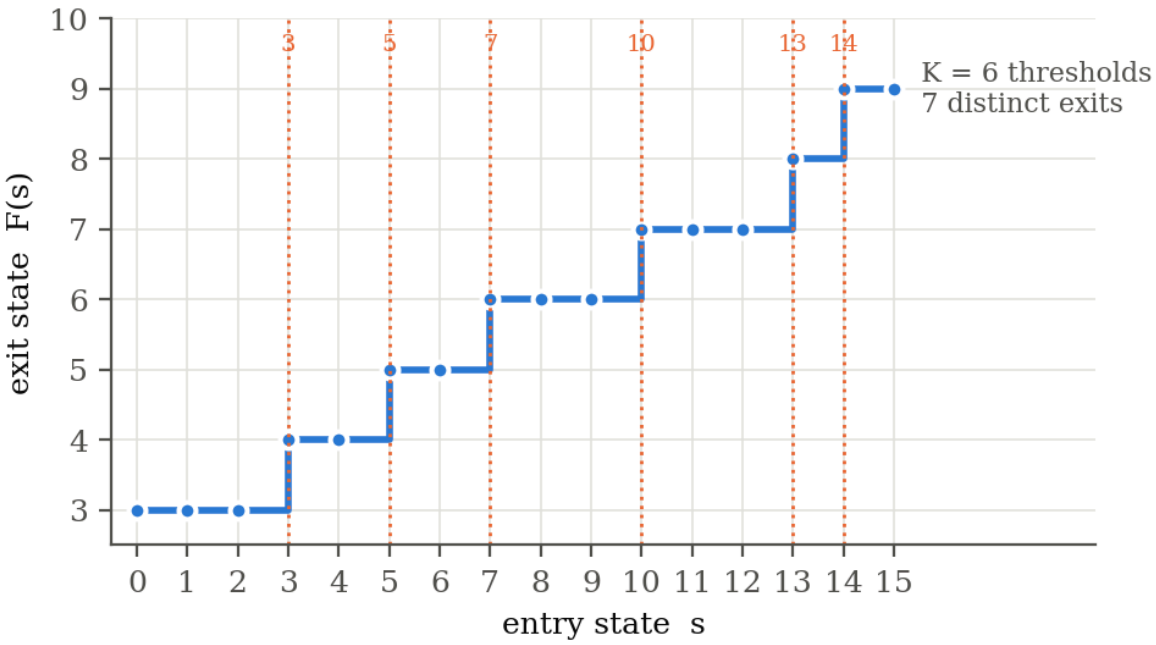


Figure 1.3. The block map of Example 1.3. Each dot is one entry state; the dotted lines mark the six thresholds where the staircase rises by one.

The cost of storing, building and querying a threshold list grows with K. So the central question becomes: **how small does K get?**

1.6 How far can a block collapse?

1.6.1 The width recurrence

Start with every possible entry, $\{0, \dots, S_{\max}\}$, a set of width $w_0 = S_{\max}$. After each step, apply (1.4) to the smallest and largest surviving states. The remainder can be as large as $D - 1$, so in the worst case

$$w' \;\le\; w + \left\lfloor \frac{D - 1 - mw}{D} \right\rfloor = \left\lfloor \frac{Aw + D - 1}{D} \right\rfloor = \left\lceil \frac{Aw}{D} \right\rceil. \tag{1.11}$$

(The middle step writes $w = Dw/D$ and $-mw = Aw - Dw$; the last uses $\lfloor (N + D - 1)/D \rfloor = \lceil N/D \rceil$ for integers N.)

While w is large, the ceiling hardly matters and the width shrinks by the factor a per step, like the real filter. But the ceiling stops the shrinking once $\lceil Aw/D \rceil = w$. Since $Aw/D \le w$, this happens exactly when $Aw/D > w - 1$, that is $mw < D$. The largest width that can stop is therefore

$$R = \left\lfloor \frac{D - 1}{m} \right\rfloor. \tag{1.12}$$

For 63/64, $R = 63$. For 61/64, $R = \lfloor 63/3 \rfloor = 21$.

1.6.2 The floor is real

Equation (1.11) is an upper bound. Is it ever reached? Yes: the constant-input band (1.7) contains about $\tau = D/m$ stuck states that never move, so no block length can collapse them. For 63/64 a constant input keeps exactly $64 = R + 1$ distinct exits, so $K = R$. **The bound is tight. R is the true worst case over all inputs, and no block length can promise anything smaller.**

Let $H(B)$ denote the number of iterations of (1.11) needed to bring $S_{\max}$ down to R. Iterating it directly:

Table 1.3. The width floor R and the number of steps H to reach it from a full 17-bit range.

Retention a	m	τ	R	H **at** $B = 17$
3/4	1	4	3	39
15/16	1	16	15	149
61/64	3	21.3	21	**194**
63/64	1	64	63	**521**

> **Why the estimate $L \approx \tau \ln(wo/K)$ misleads.** That formula assumes the width shrinks like $w_0 a^L$ forever. It is accurate while $w \gg \tau$, and for a target of $K = R$ it gives a reasonable length. But ask it for $K < R$ and it returns a finite block length that does not exist: a constant input never collapses below R, however long the block.

1.6.3 Varying inputs collapse further

The floor R is a property of *steady* inputs. Return to (1.5). With a constant input, the stuck states never move, so their remainders never change, and a gap that survived once survives forever. With a varying input, c changes every step, so the remainders change; sooner or later one lands below md and the gap closes. Figure 1.4 shows measured collapse: up to about 500 steps every input behaves the same, then steady inputs halt at their floor while random inputs keep collapsing, down to a single state.

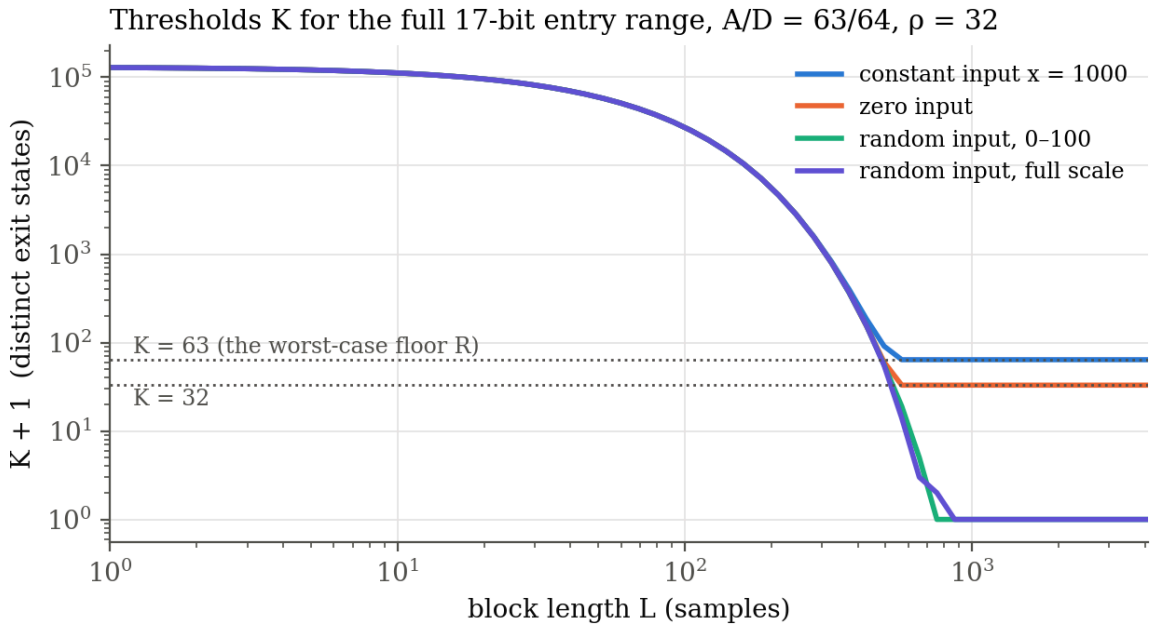


Figure 1.4. Distinct exit states K + 1 after a block of length L, computed from all 131,072 entries (a = 63/64, ρ = 32). Constant input stops at 64 (K = 63 = R); zero input stops at 33 (the dead band). Random inputs collapse to one state.

The practical consequence is simple and important: **hardware must be sized for steady inputs.** Silence and DC are the stress tests, not noise.

1.7 Algorithm I: the threshold-map executor

We now have enough to build a first parallel algorithm. It is worth studying carefully even though it turns out to be slow, because seeing exactly why it is slow points directly at the fix.

1.7.1 Three phases

Cut the stream into blocks of length L and give P consecutive blocks to P workers.

1. **Build (parallel).** Each worker computes its block map in threshold form (1.10).
2. **Carry (serial).** Starting from the known $s[0]$, look up each block's exit in turn: $e_{j+1} = F_j(e_j)$.
3. **Replay (parallel).** Each worker reruns the plain filter from its now-known entry e_j to produce its L outputs.

1.7.2 Building the threshold list

How does a worker find the thresholds without evaluating F at all 2^B entries? It runs the block from the two extremes, $\underline{s} = F(0)$ and $\bar{s} = F(S_{\max})$, which gives the range of exits and hence $K = \bar{s} - \underline{s}$. Then, for each target exit $t \in \{\underline{s}+1, \ldots, \bar{s}\}$, it walks the block **backward** to find the smallest entry that reaches at least t. For one step, the smallest s with $\lfloor (As + c)/D \rfloor \geq t$ satisfies $As + c \geq Dt$, so

$$s_{\min}(t) = \left\lceil \frac{D\,t - c}{A} \right\rceil, \tag{1.13}$$

clipped to $[0, S_{\max} + 1]$. Applying (1.13) from the last input back to the first gives one threshold per backward pass.

1.7.3 What it costs

Per block, a worker makes one forward pass for the extremes (both run together), K backward passes, and one replay pass: about $W = K + 3$ passes of L steps each, counting a little overhead. With P workers sharing the work, the throughput is

$$\Theta_{\text{K-map}} \approx \frac{P}{K+3} \text{ samples per clock.} \tag{1.14}$$

And now the floor from Section 1.6 bites. A design must handle the worst input, so $K = R$:

a	R	$W = R + 3$	Θ **with** $P = 8$
61/64	21	24	0.333
63/64	63	66	0.121

Both are **below 1**. Eight workers running the threshold-map algorithm are slower than one serial filter. Making blocks longer does not help, because K cannot go below R. Two further costs make it worse: the backward step (1.13) divides by A, which is not a power of two, and a lookup against K thresholds needs K comparators.

> **Diagnosis.** The threshold map prepares for *every* entry from 0 to $S_{\max}$, and pays for each surviving threshold with a full pass. But in a real stream, the entry to block j is not an arbitrary number. It is the result of filtering everything that came before, and the inputs just before the block are already known. Section 1.8 asks: how much does that knowledge narrow down the entry?

1.8 A change of viewpoint: certifying the entry

1.8.1 Rails

Let block j start at sample index n_0, and let its true entry be $e = s[n_0]$. We do not know e yet, because it depends on blocks still being processed. But we *do* know the H inputs $x[n_0 - H + 1], \ldots, x[n_0]$ immediately before the block. (In this chapter "the history of block j" means these H samples.) Whatever the state was at index $n_0 - H$, it lay somewhere in $[0, S_{\max}]$. So run two copies of the filter through the history:

- the **lower rail** ℓ, started at 0, and

- the **upper rail** h, started at $S_{\max}$.

By Fact 1 (monotonicity), the true trajectory starts between them and can never cross either one. At the end of the history,

$$\ell \;\le\; e \;\le\; h. \tag{1.15}$$

This is the *certificate*. It is not a guess or a prediction: it holds for every possible input stream, because it uses only monotonicity. And by the width recurrence (1.11), the window it certifies has width $h - \ell \le R$ once $H \ge H(B)$.

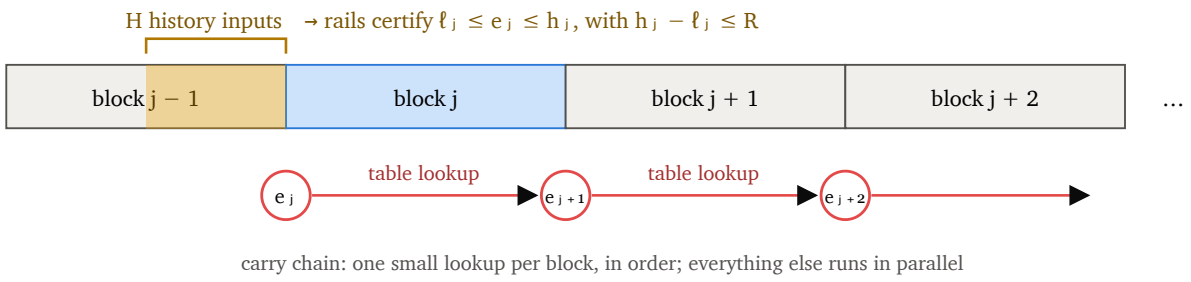


Figure 1.5. The stream view of H-cert. The last H inputs before block j (shaded) are pushed through two rails, which bracket block j's unknown entry inside a window of width at most R. Each block builds a small table for that window, and the true entries are then chained through the tables one lookup at a time.

Figure 1.6 shows the rails on real data. They begin 131071 apart and close in exponentially; the true trajectories, whatever their starting points, are trapped between them.

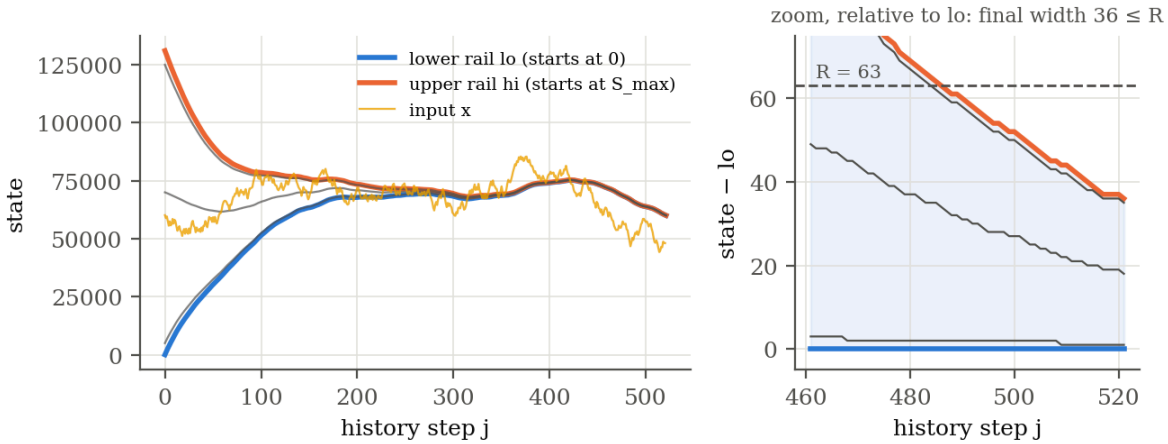


Figure 1.6. Rails over H = 521 history steps for a = 63/64, ρ = 32, with a random-walk input (yellow). Grey: true trajectories from three different unknown starting states. Right: the last 60 steps, plotted relative to the lower rail. The certified window has width 36 ≤ R = 63, and every true trajectory lies inside it.

1.8.2 How much history is enough?

The certificate needs the window to be no wider than R, for a reason that will become clear in Section 1.9. The width recurrence (1.11) gives a guaranteed history length. Its worst case is reached by constant input, so $H = H(B)$ from Table 1.3 is both sufficient and, for a guarantee, necessary: 194 samples for 61/64 and 521 for 63/64. Figure 1.7 compares the bound with an actual pair of rails. Actual rails are usually narrower than the bound, but steady inputs reach it exactly.

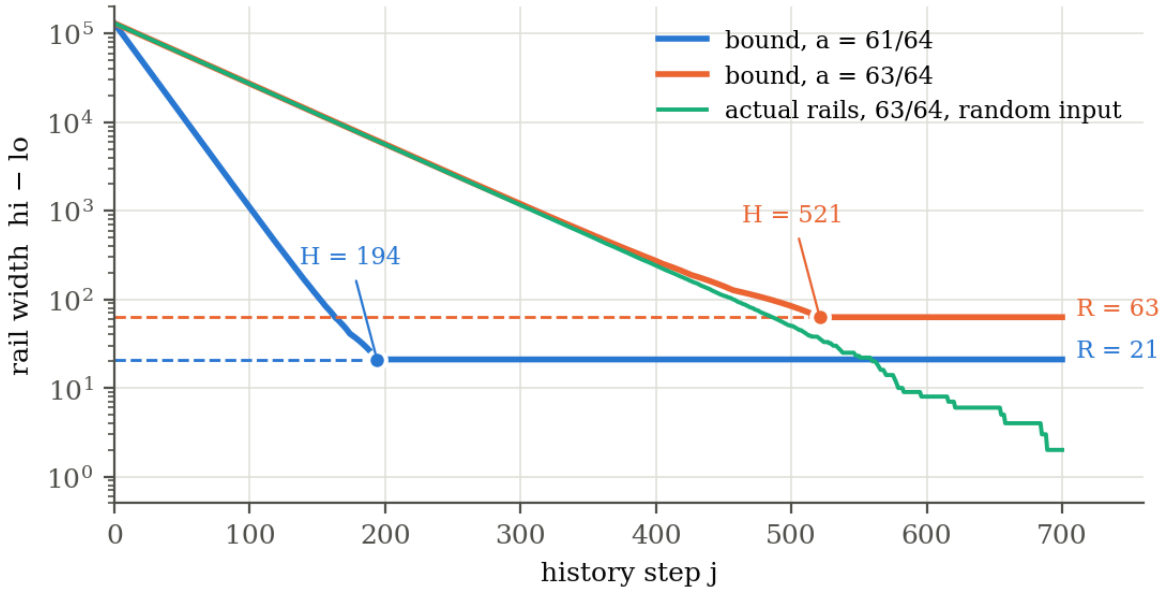


Figure 1.7. The width bound from equation (1.11) for a = 61/64 and 63/64 at B = 17, reaching R at H = 194 and H = 521. Green: the actual rail width for a random input at 63/64, which falls below R and keeps collapsing.

Example 1.4 — Rails by hand. Toy filter 3/4, $\rho = 2$, $B = 4$. Here $R = 3$, and iterating (1.11) from 15 gives $15, 12, 9, 7, 6, 5, 4, 3$, so $H = 7$. Take history inputs $9, 0, 4, 12, 7, 7, 2$.

step	0	1	2	3	4	5	6	7
input x	—	9	0	4	12	7	7	2
lower rail ℓ	0	2	2	3	5	6	6	**5**
upper rail h	15	14	11	9	10	9	9	**7**
width $h - \ell$	15	12	9	6	5	3	3	2
bound (1.11)	15	12	9	7	6	5	4	3

The certificate is $5 \le e \le 7$. To check it, run the history from all 16 possible starting states: the set of states reached is exactly $\{5, 6, 7\}$. The rails are not just safe here, they are tight.

What happens at the start of the stream? Block 1 enters at the known initial state $s[0]$, so its window is $[s[0], s[0]]$, of width 0. Any block whose history would reach back before sample 1 starts both rails at $s[0]$ instead of at 0 and $S_{\max}$, and obtains an exact entry. In both cases the certificate (1.15) still holds.

1.9 The shared-base rank table

Here is where we stand. The rails of Section 1.8 have told us that block j's true entry is one of the $R+1$ states $\ell, \ell+1, \ldots, \ell+R$. We don't yet know which one, because that depends on blocks still being processed. So the plan is to work out the exit state for **every** candidate now. When the true entry arrives later, we just look up the matching answer.

This section builds the data structure that holds those $R+1$ answers. We take it in small steps: first the obvious approach and what it costs, then the one idea that makes it cheap, then a careful look at a single update, and finally the full table and how to read it.

1.9.1 The obvious approach, and why it is too big

The simplest way to get $R+1$ answers is to build $R+1$ copies of the filter. Copy i starts at $\ell+i$, and every copy reads the same block inputs. After L steps, copy i holds $F(\ell+i)$.

This works, but it is expensive. Each copy is a full B-bit filter with its own adders and register, and there are 22 copies for 61/64 and 64 copies for 63/64, in every worker.

Notice, though, that the copies are wasting most of their effort. They start at neighbouring values, and by Fact 2 they can never drift further apart. At the start of the block, copies 0 and R hold numbers like $40,000$ and $40,021$: all their high bits are identical, and they stay identical. Almost all of the work is repeated.

1.9.2 One leader, many followers

Think of a column of cars driving down a highway. To say where every car is, you don't need each car's full mile marker. Give the lead car's position in full, and describe every other car by how many car lengths it is behind the leader. The leader's position is a big number; the distances are small.

We do the same thing with the filter copies.

- The **leader**, or *base*, is a full-width filter that starts at the bottom of the window: $b = \ell$.
- Each **follower** i (for $i = 0, 1, \ldots, R$) stands for the copy that started at $\ell+i$. We do not store its state. We store only its *offset* u_i, its distance above the leader. The follower's actual state is then $b + u_i$.

At the start of the block every follower is exactly i above the leader, so $u_i = i$. (Follower 0 is the leader itself, so $u_0 = 0$ always.)

Two facts we already proved keep every offset small:

- By Fact 1, a follower that starts above the leader never falls below it, so $u_i \geq 0$.
- By Fact 2, distances never grow, so $u_i \leq i \leq R$.

So each offset always fits in a few bits: 5 bits for 61/64 ($R = 21$), 6 bits for 63/64 ($R = 63$). The leader is the only number that needs the full B bits.

That leaves one question, and it is the heart of this section: **when the leader takes a step, how does a follower's offset change, if we never compute the follower's full state?**

1.9.3 What happens to one offset in one step

We answer the question in three short steps. Throughout, $c = m\,x + \rho$ is the constant for the current input, the same for the leader and every follower.

Step A: the leader's division leaves a leftover

The leader computes $Ab + c$ and divides by D. Like any whole-number division, this has a quotient and a leftover:

$$A\,b + c_n = D\,b' + r, \qquad 0 \leq r \leq D - 1. \tag{1.16}$$

The quotient b' is the leader's new state. The leftover r is the part the floor throws away. We call it the **remainder**. In hardware it costs nothing extra: since $D = 64$, r is simply the low 6 bits of the number $Ab + c$ that the leader computes anyway, and b' is the rest.

Another way to say this: the leader's exact, unrounded new value is $b' + r/D$. The remainder tells us how far past the integer b' that exact value landed, measured in steps of $1/D$.

Step B: the follower's exact value is the leader's, shifted

A follower with offset u is at state $b + u$. Its numerator is

$$A\,(b + u) + c = (A\,b + c) + A\,u = D\,b' + r + A\,u.$$

Now use $A = D - m$, so $Au = Du - mu$:

$$A\,(b + u) + c = D\,(b' + u) + (r - m\,u).$$

Dividing by D, the follower's exact new value is

$$\underbrace{b' + \frac{r}{D}}_{\text{leader's exact value}} \;+\; u \;-\; \underbrace{\frac{m\,u}{D}}_{\text{the pull}}.$$

In words, the follower lands u above the leader, except that it is pulled back by a small amount mu/D. That pull is just the filter doing its job: every step, it moves every state a fraction $1/\tau = m/D$ of the way toward the input, so any distance u wants to shrink by $u/\tau = mu/D$.

How big is the pull? Since $u \leq R$ and $R = \lfloor (D - 1)/m \rfloor$, we get $mu \leq D - 1$, so the pull is **less than one whole unit**. That fact is what makes everything below so simple.

Step C: rounding decides whether the pull costs a whole unit

The hardware stores whole numbers, so the follower's exact value gets rounded down. The pull is less than one unit. The only question is whether it drags the follower's exact value below the integer $b' + u$.

Picture the leader's exact value $b' + r/D$. It sits a distance r/D above the integer b'. The follower's exact value starts at the matching spot, r/D above $b' + u$, and is then pulled down by mu/D. Figure 1.8 shows the two cases.

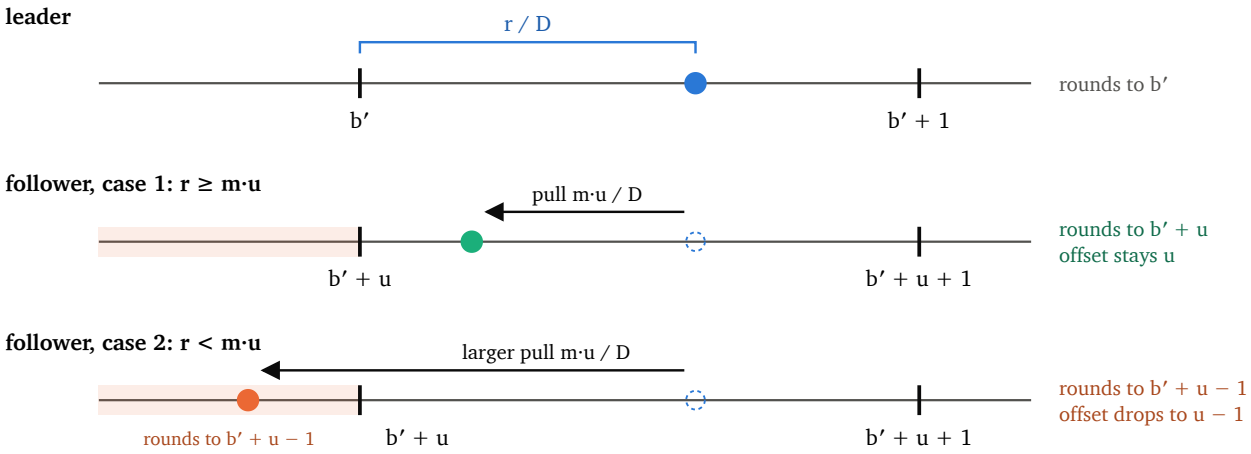


Figure 1.8. Why one comparison decides each follower. Top: the leader's exact value lands r/D above the integer b'. Each follower's exact value starts at the matching spot above b' + u (dashed circle) and is pulled left by m·u/D, which is less than one unit. Case 1: the pull is no bigger than the leftover, so the follower stays at or above b' + u and its offset is unchanged. Case 2: the pull is bigger than the leftover, so the follower crosses into the shaded zone and rounds down one further, and its offset drops by one. Because the pull is under one unit, it can never cross two integers.

- **Case 1,** $r \geq mu$**.** The leftover is big enough to absorb the pull. The follower's exact value stays at or above $b' + u$, it rounds to $b' + u$, and its offset is still u.
- **Case 2,** $r < mu$**.** The pull is bigger than the leftover. The follower dips just below $b' + u$, rounds down to $b' + u - 1$, and its offset drops to $u - 1$.

Because the pull is less than one unit, those are the only two possibilities: the follower can never drop two places. Written as a formula, the exact update is

$$u' = u + \left\lfloor \frac{r - m\,u}{D} \right\rfloor, \tag{1.17}$$

which is the gap formula (1.4) from Section 1.3 with $d = u$. Since the pull is less than one, the floor is only ever 0 or -1, and the update reduces to a single comparison:

$$u_i' = u_i - [\,r < m\,u_i\,]. \tag{1.18}$$

Here $[\,\cdot\,]$ means 1 if the condition is true and 0 if it is false. Nothing has been approximated. A follower updated this way holds, bit for bit, the same state as a full-width copy of the filter would.

Example 1.5 — One step, both cases. Take $a = 61/64$ ($A = 61$, $D = 64$, $m = 3$), $\rho = 32$, leader $b = 1000$, input $x = 1200$, so $c = 3 \cdot 1200 + 32 = 3632$.

Leader. $Ab + c = 61000 + 3632 = 64632 = 64 \cdot 1009 + 56$. So $b' = 1009$ and $r = 56$.

Follower with $u = 18$. The pull is $mu = 54$ sixty-fourths. Since $r = 56 \geq 54$, this is case 1, and the offset stays 18, so the follower is at $1009 + 18 = 1027$. Check with the full filter: $61 \cdot 1018 + 3632 = 65730$, and $\lfloor 65730/64 \rfloor = 1027$. ✓

Follower with $u = 20$. The pull is $mu = 60$. Since $r = 56 < 60$, this is case 2, and the offset drops to 19, so the follower is at $1009 + 19 = 1028$. Check: $61 \cdot 1020 + 3632 = 65852$, and $\lfloor 65852/64 \rfloor = 1028$. ✓

The leader did all the 17-bit arithmetic. Each follower only compared two 6-bit numbers.

This is why the window must be at most R wide. The single comparison works only because the pull mu/D is less than one unit, and that needs $mu \leq D - 1$, which is exactly $u \leq R$. If the certificate allowed a wider window, some follower could start with $u > R$. Its pull could then exceed one unit, the floor in (1.17) could be -2, and one comparison would no longer be enough (Problem 9 has a concrete case). The rails of Section 1.8 guarantee the width is at most R, so this never happens.

1.9.4 The whole table

Now put everything together. A worker keeps one leader and $R + 1$ followers, all reading the same block inputs. At every step:

1. The leader computes $Ab + c_n$ and splits it into the new leader b' (high bits) and the remainder r (low bits).
2. The remainder is sent to every follower at once.
3. Each follower compares r with mu_i and, if r is smaller, subtracts one from its offset.

After the L block inputs, the worker holds the leader b and the offsets $u_0, \ldots, u_R$, with

$$b + u_i = F(\ell + i) \qquad \text{for } i = 0, 1, \ldots, R.$$

We call this pair, one shared leader plus a list of small offsets, the **shared-base rank table**. Figure 1.9 shows it as hardware. In circuit terms, followers are called *lanes*, and the leader is called the *base*.

Why this is cheap. There is one full-width datapath per worker: the base. Each lane is a tiny register (5 or 6 bits), one comparator against the 6-bit remainder, and a decrement. One more trick saves logic: store $v_i = m\,u_i$ instead of u_i. The test becomes $r < v_i$, and the decrement subtracts the constant m, so no multiplier is needed. Overall, one wide datapath plus $R + 1$ small comparators replaces the $R + 1$ wide datapaths of Section 1.9.1.

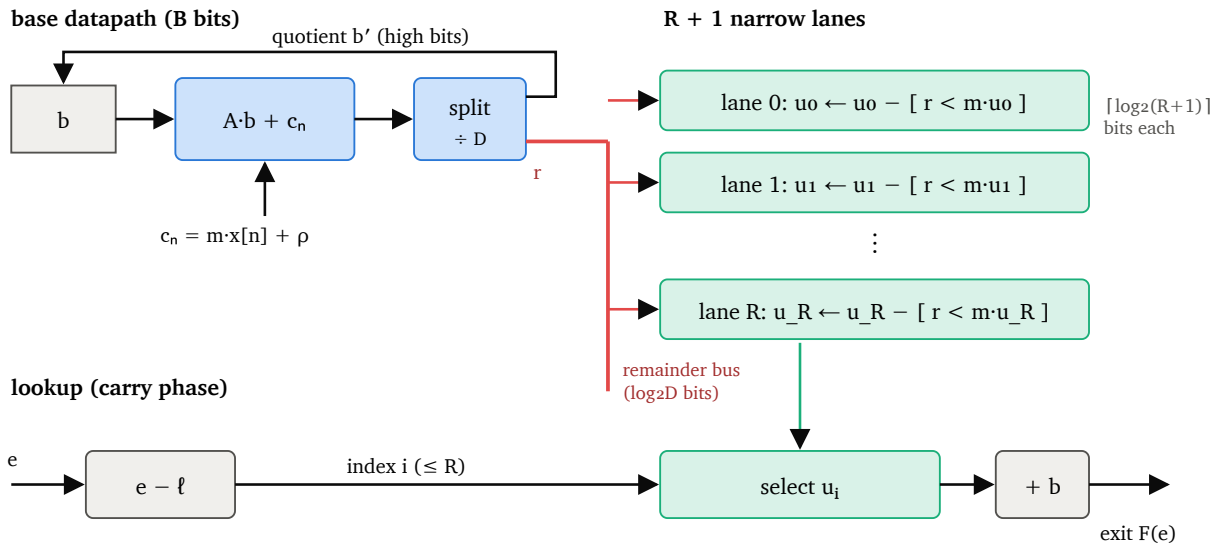


Figure 1.9. The shared-base rank table. One full-width base datapath computes b' and the remainder r for each input. The remainder is broadcast to R + 1 narrow lanes, each of which decrements its offset when $r < m \cdot u_i$ (equation (1.18)). After the block, the exit for any certified entry e is b + u at index $e - \ell$.

1.9.5 Reading the table

Later, the carry chain delivers the true entry e. The certificate promises $\ell \le e \le \ell + R$, so e is follower number $e - \ell$, and its exit is

$$F(e) = b + u_{e-\ell}. \tag{1.19}$$

That is one small subtraction (to find the follower number), one selection out of $R + 1$ offsets, and one addition. Compare that with the threshold map of Section 1.7, which needed K comparators for every lookup.

Example 1.6 — A complete table by hand. Continue Example 1.4 with the toy filter 3/4 ($m = 1$, $\rho = 2$, so $c = x + 2$). The rails certified $\ell = 5$, $h = 7$, and $R = 3$, so we keep four followers for entries $5, 6, 7, 8$. Follower 3 (entry 8) is outside the certified window. It is built anyway, because hardware always provides $R + 1$ lanes, but it will never be selected. Block inputs: $10, 3, 3, 0$.

n	x	c	$Ab + c$	**leader** b'	**remainder** r	**offsets** $u_0..u_3$	**follower states** $b + u_i$
0	—	—	—	5	—	0, 1, 2, 3	5, 6, 7, 8
1	10	12	27	6	3	0, 1, 2, 3	6, 7, 8, 9
2	3	5	23	5	3	0, 1, 2, 3	5, 6, 7, 8
3	3	5	20	5	0	0, 0, 1, 2	5, 5, 6, 7
4	0	2	17	4	1	0, 0, 1, 1	4, 4, 5, 5

Read each row with the rule "drop by one if $r < u$" (here $m = 1$):

- *Steps 1 and 2:* $r = 3$, and no offset is bigger than 3, so nobody moves.
- *Step 3:* $r = 0$. Every offset bigger than 0 drops by one.
- *Step 4:* $r = 1$. Only offsets bigger than 1 drop, and only follower 3 has one.

Now suppose the carry chain delivers $e = 7$. That is follower $7 - 5 = 2$, so $F(7) = b + u_2 = 4 + 1 = 5$. Check by running the plain filter from 7 through $10, 3, 3, 0$: it goes $8, 7, 6, 5$ and ends at 5. ✓

1.9.6 Watching a real table

Figure 1.10 shows the followers of a real 61/64 table, all 22 of them, during one block. Each row is one follower and the colour is its offset.

- With a **sine input**, the input changes every step, so the remainder r jumps around. Sooner or later it lands below each follower's pull, and the offsets drop one by one until every follower has merged with the leader.
- With a **constant input**, the leader settles, the remainder stops changing, and each follower gets the same verdict every step. A follower that survived once survives forever. This is the floor of Section 1.6, now visible as hardware state.

That is why the hardware must always provide all $R + 1$ lanes: a steady input keeps nearly all of them alive.

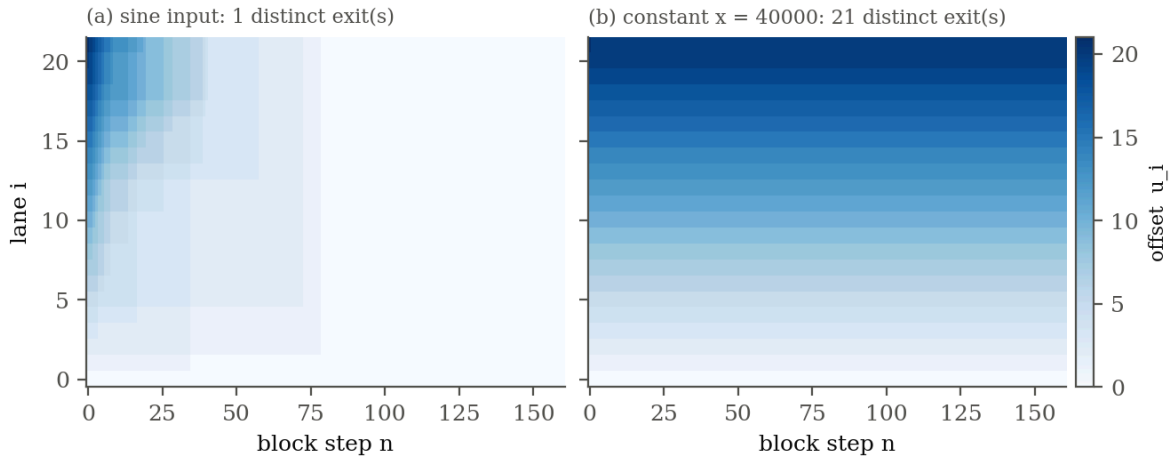


Figure 1.10. Follower offsets of a 61/64 rank table (22 lanes) during a block. (a) Sine input: the offsets collapse to a single exit. (b) Constant input: the remainder never changes, so the offsets never change, and 21 distinct exits survive.

1.10 Algorithm II: the H-cert executor

1.10.1 The algorithm

We can now assemble the complete executor. Groups of P consecutive blocks are handed to P workers.

```
(lo, hi denote the rails ℓ, h of the text)
for each group of P blocks, block j handled by worker j:

  1. CERTIFY  (parallel, H clocks)
       (lo_j, hi_j) ← run lower rail from 0 and upper rail from S_max
                      through the H inputs before block j
       [first blocks: start both rails at the known s[0]]

  2. BUILD    (parallel, L clocks)
       b ← lo_j ;  u_i ← i  for i = 0..R
       for each input x of block j:
           b, r ← divmod(A·b + m·x + ρ, D)
           for each lane i:  if r < m·u_i:  u_i ← u_i − 1

  3. CARRY    (serial, 1 clock per block)
       e_{j+1} ← b_j + u_j[ e_j − lo_j ]       (e_1 = s[0] for the first group)

  4. REPLAY   (parallel, L clocks)
       run the plain filter through block j from e_j and emit all L outputs
```

1.10.2 Why it is exact

The argument is a short induction over blocks. Suppose the entry e_j delivered by the carry chain is the true state at the start of block j (true for $j = 1$, since $e_1 = s[0]$).

1. *Certificate.* The true state H samples before block j lies in $[0, S_{\max}]$. The rails start at the two ends of that range and see the same inputs, so by monotonicity $\ell_j \le e_j \le h_j$. By (1.11) and the choice $H \ge H(B)$, $h_j - \ell_j \le R$. So $e_j - \ell_j$ is a valid lane index.
2. *Table.* The base is an ordinary run of the filter from ℓ_j. By the identity (1.17), which is exact, lane i equals the state of an ordinary run from $\ell_j + i$. Since $u_i \le R$ throughout, the comparison rule (1.18) is the same as (1.17).
3. *Carry.* Hence $b_j + u_{e_j - \ell_j}$ is exactly the state reached from e_j at the end of block j, which is the true entry e_{j+1} of block $j+1$.
4. *Replay.* The replay is the original filter run from the true entry, so its outputs are the true outputs.

Nothing in the argument depends on the input values, so the result holds for every input stream. There is no speculation and no repair path. A reference model of this algorithm, checked against the serial filter on constant, zero, random, sinusoidal and step inputs with several rounding offsets and block lengths, matched the serial filter bit for bit.

1.10.3 Scheduling the phases

Figure 1.11 compares the schedules. The threshold-map executor, panel (a), spends $K + 3$ full passes per block. H-cert, panel (b), spends H clocks on rails, L on the build, P on the serial carry (one lookup per block), and L on replay. In panel (c), each worker has a separate replay datapath, so replay of one group overlaps the certify and build phases of the next.

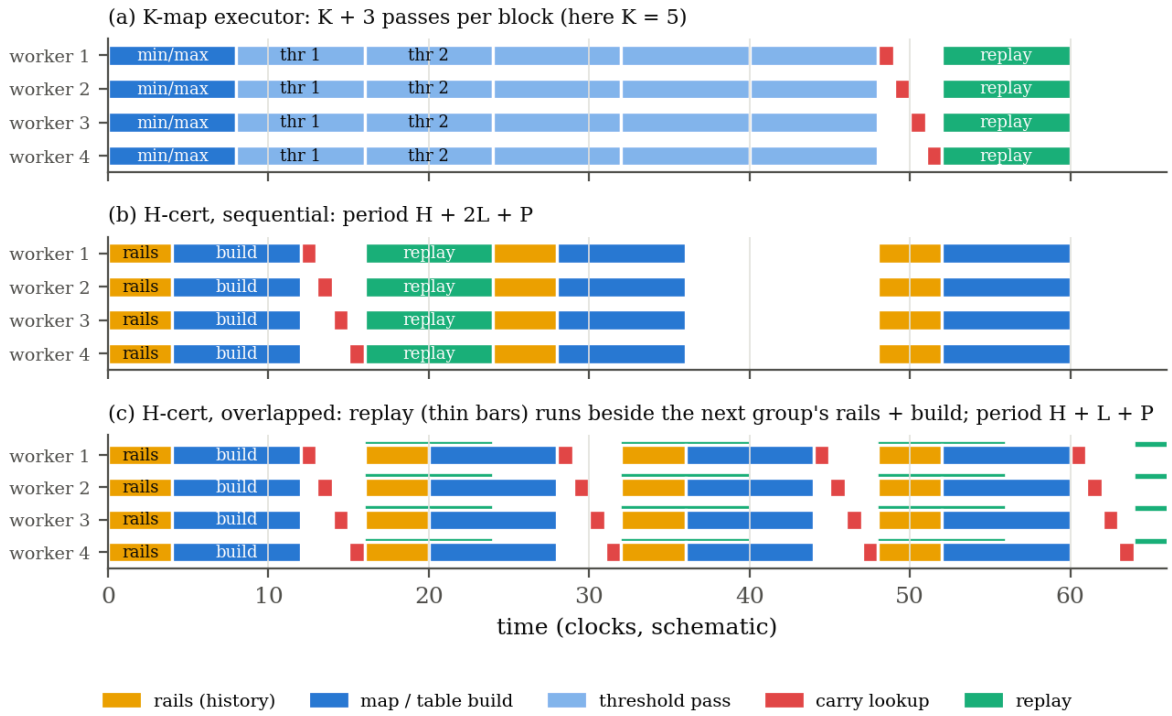


Figure 1.11. Schematic schedules for P = 4 workers. (a) The threshold-map executor needs a forward pass plus K backward passes per block before the carry chain can run. (b) H-cert, sequential: certify, build, carry, replay. (c) H-cert, overlapped: replay (thin bars) runs alongside the next group's certify and build.

1.11 Throughput and latency

1.11.1 Throughput

A group delivers PL samples. Reading the period off Figure 1.11,

$$\Theta_{\text{seq}} = \frac{PL}{H + 2L + P}, \qquad \Theta_{\text{ovl}} = \frac{PL}{H + L + P} \quad \text{samples per worker clock.} \tag{1.20}$$

Two limits are worth remembering. As $L \to \infty$, $\Theta_{\text{seq}} \to P/2$ and $\Theta_{\text{ovl}} \to P$: long blocks amortize the history, and then the replay is the only extra work. And because H enters only additively, the cost of the certificate is the fraction H/L. Compare (1.14), where the floor R divides the throughput.

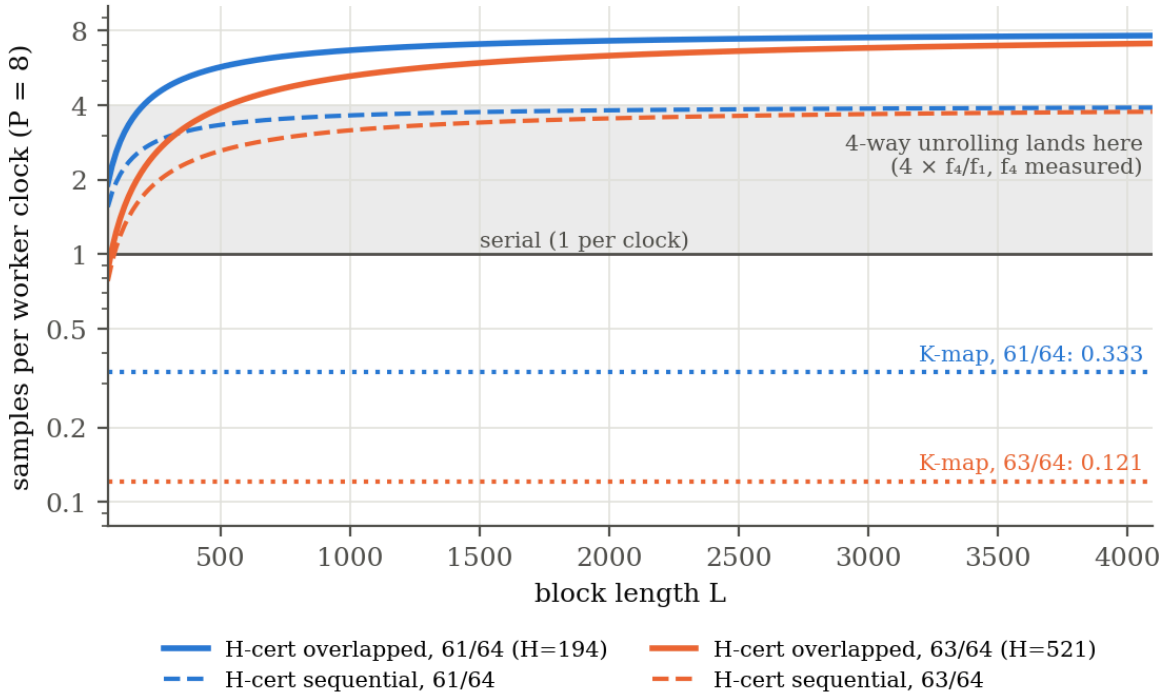


Figure 1.12. Throughput from equation (1.20) with P = 8. Solid: overlapped; dashed: sequential; dotted: the threshold-map executor. The grey band is where 4-way unrolling falls once its slower clock is accounted for: 4 f_4/f_1 samples per serial clock, somewhere between 1 and 4, to be measured.

Table 1.4. Samples per worker clock, from equations (1.14) and (1.20).

a	P	L	Θ_{seq}	Θ_{ovl}	**K-map**	**H-cert vs K-map**
61/64	8	1024	3.64	6.68	0.333	11–20×
61/64	4	256	1.44	2.26	0.167	9–14×
63/64	8	1024	3.18	5.27	0.121	26–44×
63/64	4	256	0.99	1.31	0.061	16–22×

1.11.2 Comparing with unrolling fairly

Samples per clock are only half the story, because different designs run at different clocks. The quantity that matters is samples per second:

$$\text{throughput} = \Theta \times f_{\text{clk}}. \tag{1.21}$$

A k-way unrolled design has $\Theta = k$ but clock $f_k < f_1$. A worker's critical path is roughly one update (the base, rails and replay are each one update per clock, and the lanes are shallower), so a worker should clock near f_1. H-cert at $\Theta = 2.26$ beats 4-way unrolling exactly when $2.26 f_{\text{H}} > 4 f_4$, that is when $f_4/f_{\text{H}} < 0.565$. Whether that holds is an empirical question about a specific device, and it is the experiment this chapter sets up. It cannot be settled on paper.

1.11.3 Latency

A sample waits for its whole group to arrive, then for the certify, build, carry and replay phases. If inputs arrive at Θ samples per clock, a group takes PL/Θ clocks to arrive, so a conservative bound on latency is

$$T_{\text{lat}} \lesssim \frac{PL}{\Theta} + H + 2L + P \quad \text{clocks.} \tag{1.22}$$

For 61/64, $P = 4$, $L = 256$ and $\Theta = 2.26$: about $453 + 194 + 512 + 4 \approx 1160$ clocks, roughly $23\,\mu$s at 50 MHz. That is irrelevant for monitoring or logging, and far too long for a control loop. Know which kind of consumer you are serving.

1.12 The hardware budget on a small FPGA

Throughput formulas assume the hardware exists. On a small device such as the Lattice iCE40 UP5K (5,280 four-input LUTs, 8 DSP blocks, 30 block RAMs of 4 Kbit, and 4 single-port 256-Kbit SPRAMs, each 16 bits wide), the budget decides which point on Figure 1.12 is reachable. The estimates below are first-order, meant to orient a design. Synthesis must replace them.

1.12.1 Logic

Each worker needs full-width update units for the two rails, the base, and (in the overlapped schedule) the replay, plus $R + 1$ lanes. In the sequential schedule the phases never overlap, so two full-width units can be time-shared among all of these roles. A constant-coefficient update is two or three adders of about $B + 6$ bits, so roughly 60–80 LUTs. A lane is a 6-bit compare plus a small subtract and register, perhaps 12–15 LUTs.

Table 1.5. Rough logic per worker. With 5,280 LUTs, 61/64 admits about 4–6 workers and 63/64 about 3. A 4-way unrolled baseline is about four update units, a few hundred LUTs.

Item	61/64 **($R + 1 = 22$)**	63/64 **($R + 1 = 64$)**
Lanes	$\approx$ 300 LUTs	$\approx$ 850 LUTs
Full-width units (3–4)	$\approx$ 250 LUTs	$\approx$ 250 LUTs
Lookup and control	$\approx$ 100 LUTs	$\approx$ 150 LUTs
Per worker	**$\approx$ 650 LUTs**	**$\approx$ 1250 LUTs**

This is why 61/64 makes the better headline case: its lanes are a third as many.

1.12.2 Memory: the hidden budget

Every input is read up to three times: once by some worker's rails (as history), once by its own build, and once by its replay. Per output sample, the number of memory reads is

$$\frac{\text{reads}}{\text{output}} = 2 + \frac{H}{L}. \tag{1.23}$$

The overlapped schedule must also hold about three groups at once (one replaying, one building, one arriving), which is $3PL$ words. On the UP5K:

- **Capacity.** At $P = 8$, $L = 1024$ that is about 24,600 words, or 408 Kbit at 17 bits. That is more than the 120 Kbit of block RAM. SPRAM is large enough, but it offers only 4 ports of 16 bits per clock, against the roughly 15 reads per clock that (1.23) demands at $\Theta = 6.7$.
- **A point that fits.** Each 17-bit word needs two 16-bit-wide block RAMs side by side, so the 30 block RAMs act as 15 banks of 256 words. At $P = 4$, $L = 256$, the overlapped schedule needs $3 \times 4 = 12$ banks, one block per bank, and each bank serves at most one read and one write per clock (with one exception, the subject of Problem 11). That fits, at $\Theta_{\text{ovl}} = 2.26$ for 61/64.

> **The lesson.** On a small FPGA, memory capacity and ports, not arithmetic, decide the block length L, and L decides how much of the certificate cost is amortized. Budget memory first. Narrowing the word to 16 bits doubles the number of banks (and lowers H to 179 for 61/64), which roughly doubles the feasible P.

1.13 Measuring what matters

A single sample arriving per clock does not cap a parallel design's throughput, provided the interface delivers several samples per clock. To measure throughput:

1. Give every design the same W-sample-wide input and output stream with a valid/ready handshake.
2. Drive it from a testbench source that never stalls, and check every output against a golden model of the serial filter.
3. Count the clocks C needed to produce N outputs in steady state, and read the routed $f_{\max}$ from the place-and-route report. Then report

$$\text{measured throughput} = \frac{N}{C} \cdot f_{\max}, \tag{1.24}$$

alongside latency, LUTs, block RAMs and power, as separate numbers.

The comparison is fair only if every design uses the same coefficient, word width, rounding offset, device and input interface, and if the unrolled baselines get their own best clock. The stress inputs are constant and zero streams (Section 1.6).

1.14 What is new, and what is not

It is part of understanding a method to know where it came from.

- **Monotone block maps composed in parallel** are an established idea for nonlinear recurrences, for example the pipelined saturated accumulation of Papadantonakis et al.
- **Using the inputs before a chunk to restrict its possible entry states** is established in the speculation literature: restricted-entry execution of finite-state machines, and lookback.
- **The adjacent-collapse structure** of the staircase maps has been studied mathematically.
- **The specific pairing** of a numerical rail certificate with an exact remainder-driven rank table for this integer recurrence was not found in the sources reviewed, but it is elementary once the gap formula is known.

The honest claim is therefore not "a new principle". It is a **specialized exact executor with a measured operating region**: a demonstration that, on a named small FPGA, for a named coefficient and rounding, the complete circuit beats the best exact unrolled design above some input rate, with every cost included. Two readings remain before any novelty claim: Parhi's 1991 work on look-ahead for nonlinear recurrences, and the full methods of the closest speculation papers.

1.15 Summary

- **The integer update** $s' = \lfloor (As + mx + \rho)/D \rfloor$ rounds once per step. Rounding steps cannot be merged, so exact speed-up must respect each one (Example 1.1).
- **One step is monotone, and gaps obey** $d' = d + \lfloor (r - md)/D \rfloor$ (equation (1.4)). Gaps never grow. A small gap closes by one exactly when $r < md$.
- **Steady inputs freeze states.** Zero input sticks at $\lfloor \rho/m \rfloor$. A constant input keeps a band of about τ states fixed.
- **A block map is a staircase,** fully described by a base and K thresholds. For any inputs, $K \le R = \lfloor (D-1)/m \rfloor$ once $L \ge H(B)$, and constant inputs reach $K = R$.
- **The threshold-map executor** costs about $K + 3$ passes per block, so the floor R makes it slower than serial for both 61/64 and 63/64.
- **H-cert** certifies each block's entry to a window of width $\le R$ using H preceding inputs, builds an exact rank table with one wide base and $R + 1$ one-comparison lanes, chains entries with one lookup per block, and replays. Its throughput is $PL/(H + 2L + P)$, or $PL/(H + L + P)$ overlapped.
- **On a small FPGA, memory sets** L**,** and samples per second, $\Theta f_{\max}$, is the only fair figure of merit.

Table 1.6. The chapter in seven lines.

Result	Equation
Update	$s' = \lfloor (As + c)/D \rfloor$, $c = mx + \rho$
Gap	$d' = d + \lfloor (r - md)/D \rfloor$
Width bound and floor	$w' \le \lceil Aw/D \rceil$, $R = \lfloor (D-1)/m \rfloor$
Certificate	$\ell \le e \le h$, $h - \ell \le R$ after $H(B)$ steps
Lane rule	$u_i' = u_i - [\, r < m u_i \,]$
Lookup	$F(e) = b + u_{e-\ell}$
Throughput	$PL/(H + 2L + P)$, overlapped $PL/(H + L + P)$

Problems

1. **(Rounding.)** For $a = 61/64$, find the zero-input dead band for $\rho = 0$, $\rho = 32$ and $\rho = 63$.

2. **(Fixed band.)** For $a = 63/64$, $\rho = 32$ and constant input $x = 5000$, list the fixed states using (1.7). How many are there?

3. **(Gap formula.)** Toy filter 3/4, $\rho = 2$, input $x = 0$. For $s = 5$ and $t = 8$, compute r and predict t' with (1.4). Verify directly.

4. **(Look-ahead.)** With the toy filter and truncation ($\rho = 0$), find s, x_1, x_2 for which the merged formula of Example 1.1 disagrees with two steps of the filter. Explain the disagreement in terms of the remainder discarded at step 1.

5. **(Width floor.)** Compute R for $a = 97/100$ and for $a = 127/128$. Which needs more lanes, and why does D alone not decide?

6. **(History length.)** Iterate (1.11) to find $H(B)$ for $a = 3/4$ at $B = 8$, and for $a = 15/16$ at $B = 12$.

7. **(Rails.)** Toy filter 3/4, $\rho = 2$, $B = 4$. Run the rails through the history $0, 0, 0, 0, 0, 0, 0$. What window is certified? Compare its width with R, and explain using the dead band.

8. **(Rank table.)** In Example 1.6, replace the last block input 0 by 15. Recompute the final row of the table and the exit for $e = 6$.

9. **(Why width R?)** Suppose a certificate window had width $R + 1$ for $a = 61/64$, so an offset could reach $u = 22$. Find the remainders r for which (1.18) gives the wrong update, and state the correct one.

10. **(Throughput.)** For $a = 61/64$ with $P = 6$ and $L = 384$, compute Θ_{seq} and Θ_{ovl}. What value of f_4/f_{H} would make the overlapped design tie with 4-way unrolling?

11. **(Bank conflict.)** In the overlapped schedule, the rails of the first block of group $g + 1$ read the tail of the last block of group g, which is also being replayed. Propose a schedule change or buffering scheme that removes this double read.

12. **(Design.)** Using Section 1.12, choose P, L and a word width for $a = 61/64$ on the UP5K that maximizes Θ_{ovl} within both the LUT and block-RAM budgets. State every assumption.

Answers to selected problems

1. $m = 3$, so the dead band is $s \le \rho/3$: $\{0\}$, $\{0, \ldots, 10\}$, and $\{0, \ldots, 21\}$.

2. $5000 - (64 - 32)/1 < s \le 5000 + 32$, i.e. $4969 \le s \le 5032$: 64 states.

3. $As + c = 15 + 2 = 17 = 4 \cdot 4 + 1$, so $s' = 4$ and $r = 1$. With $d = 3$, $md = 3 \le D - 1$, and $r = 1 < 3$, so $d' = 2$ and $t' = 6$. Directly, $\lfloor (24 + 2)/4 \rfloor = 6$. ✓

5. 97/100: $m = 3$, $R = \lfloor 99/3 \rfloor = 33$. 127/128: $m = 1$, $R = 127$. The second needs about four times as many lanes. R depends on $D/m \approx \tau$, the memory of the filter, not on D alone.

6. 3/4 at $B = 8$: $H = 17$. 15/16 at $B = 12$: $H = 96$.

7. The lower rail stays at 0. The upper rail falls $15, 11, 8, 6, 5, 4, 3, 2$. The window is $[0, 2]$, of width $2 < R = 3$. Every state in $\{0, 1, 2\}$ is in the zero-input dead band ($\rho/m = 2$), so those states can never be told apart: the certificate cannot get narrower than the dead band.

8. At step 4, $c = 17$, $Ab + c = 15 + 17 = 32 = 8 \cdot 4 + 0$, so $b' = 8$ and $r = 0$. Offsets $0, 0, 1, 2$ become $0, 0, 0, 1$. The exit for $e = 6$ is $8 + u_1 = 8$. Directly from 6: $7, 6, 5, 8$. ✓

9. With $m = 3$, $D = 64$ and $u = 22$, the exact lane update (1.17) changes the offset by $\lfloor (r - 66)/64 \rfloor$. For $r = 0$ and $r = 1$ the fraction is $-66/64$ or $-65/64$, so the floor is -2: the offset must drop by **two**. The comparison rule (1.18) subtracts only one. For $r \ge 2$ both give -1. The single-comparison lane is exact only because the certificate keeps every offset at or below R.

10. $H = 194$: $\Theta_{\text{seq}} = 2304/(194 + 768 + 6) = 2.38$ and $\Theta_{\text{ovl}} = 2304/(194 + 384 + 6) = 3.95$. A tie requires $3.95 f_{\text{H}} = 4 f_4$, i.e. $f_4/f_{\text{H}} \approx 0.99$. Since a 4-way unrolled loop clocks well below a single update, the overlapped design would almost certainly win on clock speed. But check memory first: blocks of 384 words occupy two 256-word banks each, so $3 \times 6 \times 2 = 36$ banks are needed and only 15 exist. This point does not fit the UP5K's block RAM.